Class: SE **Subject: COA LAB** **Sem-III**

Experiment No 4

Course Outcome:CO2 **Blooms Level:L3**

Aim: Implementation of two 8-bit BCD additions with accepting input from keyboard and displaying output on monitor using INT 21H interrupts.

Abstract: Develop an 8086 assembly language program to read two 2-digit BCD (Binary-Coded Decimal) numbers from the user, add them, and display the result as a decimal value. The input should be taken in the form of two separate ASCII characters for each number, representing values from 00 to 99. The program must convert the entered characters into packed BCD format, perform the addition using BCD arithmetic (using DAA instruction for decimal adjustment), and then convert the final result into ASCII format for display. Proper use of subroutines should be demonstrated for modularity, including separate routines for input, conversion, addition, and output. The program should display meaningful prompts and show the result clearly to the user.

Sample Input:

Case 1:BCD Numbers:25 and 12
Case 2:BCD Numbers:46 and 25

Theory:

Binary-Coded Decimal (BCD) is a form of number representation in which each decimal digit is stored separately in its binary form. In packed BCD format, two decimal digits are stored within a single byte — the higher nibble holds the tens place, and the lower nibble holds the units place. This format is widely used in applications where decimal precision is required, such as financial calculations, digital clocks, and embedded systems.

When performing arithmetic operations on BCD numbers in assembly language, the processor treats them as binary values. To obtain a correct BCD result after binary addition, the DAA (Decimal Adjust after Addition) instruction is used.

The DAA instruction checks whether the result in the AL register contains a valid BCD value and adjusts it by adding 6 to the lower or upper nibble if necessary. This correction ensures that each nibble remains in the range 0000–1001 (0–9 in binary).BCD addition requires careful handling of both input conversion and post-operation adjustment to ensure the final output remains in valid decimal form.

Class: SE **Subject: COA LAB** **Sem-III**

Program:

```
.model small
.stack 100h

.data
msg1 db 'Enter first BCD number (00-99): $'
msg2 db 0Dh, 0Ah, 'Enter second BCD number (00-99): $'
msg3 db 0Dh, 0Ah, 'Sum is: $'
res db 3 dup('$') ; two digits + '$'

.code
main:
    mov ax, @data
    mov ds, ax

    ; First number
    lea dx, msg1
    call print
    call read_bcd
    mov bh, al       ; store first BCD in BH

    ; Second number
    lea dx, msg2
    call print
    call read_bcd
    add al, bh       ; add two BCD numbers
    daa              ; Decimal Adjust AL
    call bcd_to_ascii

    ; Show result
    lea dx, msg3
    call print
    lea dx, res
    call print

    ; Exit
    mov ah, 4Ch
    int 21h

; ------- Subroutines -------

; Print string at DS:DX
print:
    mov ah, 09h
    int 21h
    ret

; Read 2 ASCII digits and convert to packed BCD in AL
read_bcd:
    ; Read 1st digit
    mov ah, 01h
    int 21h
    sub al, '0'
    mov cl,4
    shl al, cl
    mov bl, al       ; upper nibble

    ; Read 2nd digit
    mov ah, 01h
    int 21h
    sub al, '0'
    or bl, al        ; combine into packed BCD
    mov al, bl
    ret

; Convert packed BCD in AL to 2 ASCII digits stored in res[]
bcd_to_ascii:
    mov ah, al
    and ah, 0F0h
    mov cl,4
    shr ah, cl
    add ah, '0'
    mov res, ah

    mov ah, al
    and ah, 0Fh
    add ah, '0'
    mov res+1, ah

    mov byte ptr res+2, '$'
    ret

end main
```

Class: SE **Subject: COA LAB** **Sem-III**

Output:

```
C:\>exp4.exe
Enter first BCD number (00-99): 12
Enter second BCD number (00-99): 23
Sum is: 35
```

Conclusion:

The experiment Two 8-bit BCD Addition was successfully carried out using 8086 assembly language. Through this program, two 8-bit BCD values were accepted from the keyboard as ASCII characters, converted into numerical form, added using standard arithmetic instructions, and the result was adjusted using the DAA (Decimal Adjust after Addition) instruction to maintain BCD correctness.

The final BCD sum was then converted back to ASCII and displayed on the monitor using INT 21H DOS interrupt services, demonstrating the interaction between the CPU, user, and display. This experiment helped students understand how to handle keyboard input, perform BCD arithmetic, and generate screen output in low-level programming. It also reinforced concepts like ASCII-to-BCD conversion, use of DOS interrupts, and handling arithmetic adjustments for BCD values.

Exercise 1: To develop an 8086 assembly language program that reads two 2-digit packed BCD numbers, performs subtraction of the second number from the first using BCD arithmetic, and displays the result in valid decimal format .

Abstract: Design and implement an 8086 assembly language program to perform the subtraction of two 2-digit decimal numbers entered by the user. The numbers should be entered as individual ASCII digits, converted into packed Binary-Coded Decimal (BCD) format, and stored for computation. The program must subtract the second BCD number from the first and adjust the result to maintain valid BCD format. It should then convert the result back to ASCII digits and display the final difference. The program should include separate subroutines for input, conversion, arithmetic operation, and output. Proper handling of BCD arithmetic using relevant instructions must be demonstrated.

Sample input:

Case 1:BCD Numbers:25 and 12

Case 2:BCD Numbers:46 and 25